COURS · LICENCE

Semestre 1

Big Data

AVEC PYTHON

Avec évaluation de compétences
et TPs

COURS ÉLABORÉ PAR

Dr. Ing. Takwa Ben Aïcha Gader

Docteur en Informatique . Ambassadrice NVIDIA

takwa.ben.aichaa@gmail.com

DONNÉES

À GRANDE ÉCHELLE



HADOOP / MAPREDUCE / SPARK

NoSQL / PYTHON

Quand les données deviennent une décision

1

Une donnée n'a de valeur que si elle éclaire une action au bon moment.



Que vaut une donnée correcte si elle arrive trop tard ?

Étape 1 — 02 h 17, M42 chauffe

2

FIL ROUGE · PARC INDUSTRIEL IoT

Une température grimpe. Cinq secondes pour produire une alerte utile.



87 °C

température en hausse

vibration +18 %



ALERTE À EXPLIQUER

Le problème n'est pas « avoir des données » : c'est décider assez vite, avec assez de confiance.

Calcul guidé — combien de données arrivent ?

3

CALCUL GUIDÉ · UNITÉS OBLIGATOIRES

On dimensionne avec des unités, pas avec l'impression que « cela fait beaucoup ».

10 000	10	500
capteurs	messages/s par capteur	octets par message

$$10\,000 \times 10 \times 500$$

50 Mo/s

100 000 événements/s

50 Mo/s décrit une contrainte ; ce nombre seul ne suffit pas à définir le Big Data.

Calcul guidé — combien faut-il réellement stocker ?

4

La rétention brute n'est que le début du dimensionnement.

1. $50\ \text{Mo/s} \times 86\,400$ → **4,32 To/j**
2. $4{,}32 \times 30$ → **129,6 To / 30 j**
3. $129{,}6 \times 3$ → **388,8 To répliqués**
4. $388{,}8 \times 1{,}25$ → **486 To planifiés**



L'échelle apparaît lorsque débit × temps × résilience × marge s'accumulent.

Les 5 V sortent du cas M42

5

Les « V » nomment des difficultés observables ; ils ne remplacent pas le dimensionnement.

1	2	3	4	5
Volume	**Vélocité**	**Variété**	**Véracité**	**Valeur**
				
486 To	50 Mo/s $p95 < 5$ s	mesures · logs maintenance · ordres	absence retard · dérive	décision effet mesuré

Les V décrivent des contraintes ; ils ne remplacent pas le dimensionnement.

Pourquoi ne pas simplement acheter un serveur plus puissant ?

Scale-up ou scale-out ? Scale-up

- plus de CPU
- plus de RAM
- stockage plus rapide
- une machine de plus en plus coûteuse
- limite physique et point de panne important



Scale-out

- ajouter des machines
- répartir données et calculs
- augmenter progressivement la capacité
- concevoir la tolérance aux pannes

Pourquoi Hadoop ?

- Hadoop est un ensemble de technologies open source pour **stocker** et **traiter** de gros volumes sur un cluster.
- Il a popularisé une architecture fondée sur des machines standards, la distribution des données et le calcul près des données.
- Aujourd'hui, il reste surtout important pour comprendre les **principes fondateurs du Data Engineering distribué**.



Ce chapitre n'est pas une collection de noms : c'est une façon de penser l'échelle.

Hadoop : la carte mentale à retenir

Si la distribution est justifiée, qui stocke, qui arbitre et qui calcule ?



HDFS

STOCKER

Où placer les blocs
et leurs copies ?



YARN

ARBITRER

Qui reçoit CPU
et mémoire ?



MapReduce

CALCULER

Comment distribuer
un calcul batch ?

Hadoop devient une architecture candidate parce que le besoin a été mesuré — jamais par réflexe.

Pourquoi HDFS?

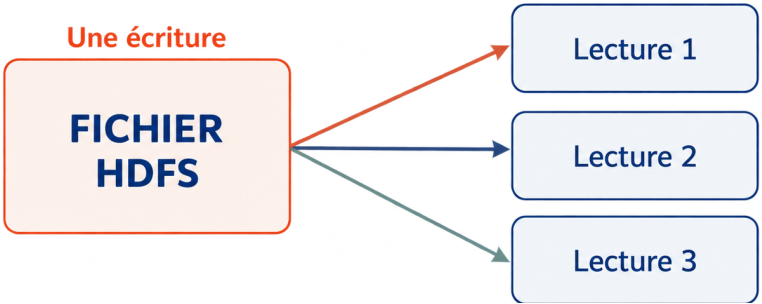
- HDFS vise les **grands fichiers** et les grands jeux de données.
- Il privilégie le **débit global** plutôt que la faible latence interactive.
- Il suppose que des disques, nœuds ou liens réseau peuvent tomber en panne.
- Il est particulièrement adapté aux traitements **batch** et aux lectures séquentielles massives.



Le stockage doit rester disponible même si une partie de l'infrastructure échoue.

Modèle d'accès : Write Once, Read Many

Un gros fichier est écrit, puis relu de nombreuses fois.



Ajout en fin possible ; petites mises à jour aléatoires non privilégiées.

Objectif : accès séquentiel et haut débit, pas latence interactive.

Un fichier HDFS devient une suite de blocs

- Un fichier est découpé en **blocs**.
- Une taille courante est **128 MiB**, mais elle est configurable.
- Les blocs permettent de répartir un fichier sur plusieurs DataNodes.
- Le dernier bloc n'occupe que l'espace réellement utilisé.



Pourquoi des blocs aussi grands ?

Réduire l'overhead de métadonnées et favoriser les lectures séquentielles.

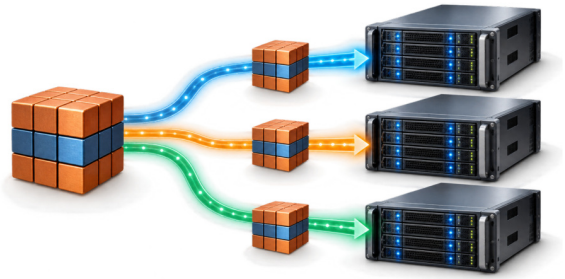
Distribution et réplication : deux idées différentes

Distribution

- blocs **différents** répartis sur plusieurs nœuds ;
- objectif : capacité et parallélisme.

Réplication

- plusieurs copies du **même bloc** sur des nœuds distincts ;
- objectif : disponibilité et tolérance aux pannes.



NameNode et DataNodes : qui fait quoi ?

NameNode

- gère l'espace de noms et les **métadonnées** ;
- connaît fichier → blocs → DataNodes ;
- coordonne les opérations de réplication.

DataNodes

- stockent réellement les blocs ;
- servent les lectures et écritures ;
- créent, suppriment et répliquent les blocs sur instruction.



Le NameNode ne transporte pas les données

Pourquoi ce choix est-il essentiel ?

- Le client demande au NameNode les **emplacements** des blocs.
- Le NameNode répond avec les métadonnées utiles.
- Le client lit ensuite **directement** auprès des DataNodes.

Le NameNode reste hors du chemin principal des données : il évite ainsi de devenir un goulot d'étranglement.



Heartbeats et Block Reports : savoir qui est vivant

- Les DataNodes envoient périodiquement des **heartbeats** au NameNode.
- Ils envoient aussi des **block reports** décrivant les blocs qu'ils hébergent.
- Si un DataNode devient indisponible, le NameNode arrête de lui confier de nouvelles E/S.
- Si la réplication tombe sous la cible, HDFS déclenche de la **re-réplication**.



Scénario de panne : un DataNode disparaît

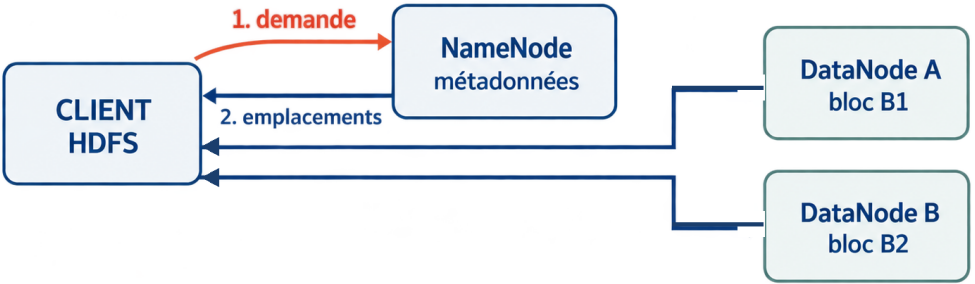
- le heartbeat s'arrête ;
- le NameNode marque le nœud indisponible ;
- les réplicas perdus ne sont plus comptés comme disponibles ;
- les blocs sous-répliqués sont recréés ailleurs.

La panne devient un événement géré par le logiciel.



Lire un fichier HDFS : qui contacte qui ?

Le NameNode localise les blocs ; le client les lit sur les DataNodes.

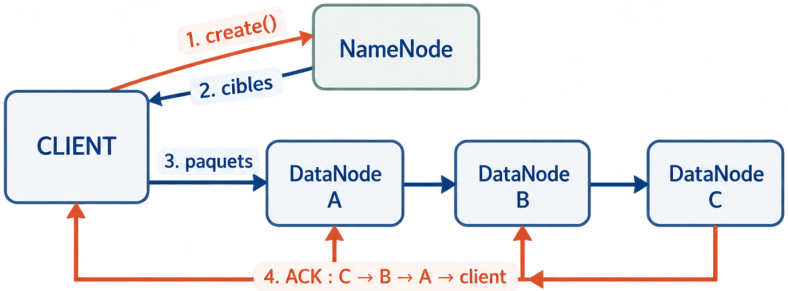


3. Lecture des blocs : flux des DataNodes vers le client

À retenir : le NameNode reste hors du transfert des blocs.

Écrire dans HDFS : pipeline et confirmations

Le client crée le fichier ; les blocs sont répliqués dans un pipeline.



Le NameNode coordonne ; les données passent par les DataNodes.

Écriture HDFS : que se passe-t-il en cas de panne ?

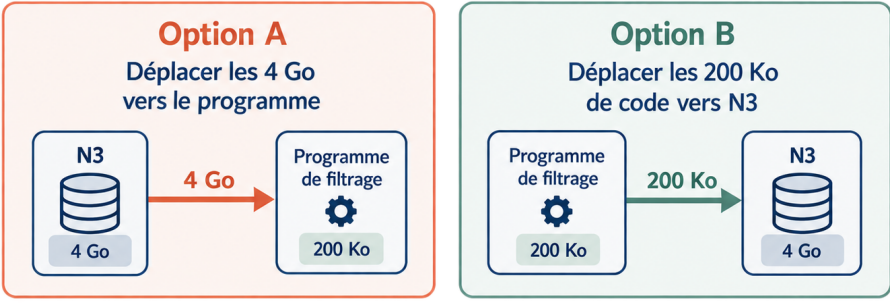
- Les paquets non acquittés restent connus du client.
- Si un DataNode du pipeline échoue, le pipeline est reconfiguré.
- La réplication cible est ensuite restaurée.
- Le NameNode coordonne les métadonnées ; les données continuent de circuler entre client et DataNodes.



Fiabilité = protocole + réplication, pas matériel supposé infaillible.

Data locality : où exécuter le calcul ?

Le bloc de 4 Go est sur N3. Le programme de filtrage fait 200 Ko.



Quelle option limite le transfert réseau ?
Principe : rapprocher le calcul du bloc de données.

Commandes HDFS : repérer et envoyer

Les chemins HDFS ne désignent pas le disque local.

1. **Lister le contenu de HDFS**

```
hdfs dfs -ls /
```

2. **Créer le dossier /data/logs**

```
hdfs dfs -mkdir -p /data/logs
```

3. **Envoyer le fichier local logs.txt vers HDFS**

```
hdfs dfs -put logs.txt /data/logs/
```

Commandes HDFS : consulter et récupérer

Suite de l'exemple : le fichier est dans /data/logs/.

4. Lire le contenu

```
hdfs dfs -cat /data/logs/logs.txt
```

5. Récupérer le fichier sur le poste local

```
hdfs dfs -get /data/logs/logs.txt ./
```

6. Supprimer le fichier dans HDFS

```
hdfs dfs -rm /data/logs/logs.txt
```

Évaluation des compétences

En classe : questions rapides ; quiz complets après la séance

Partie introductive



Ouvrir le quiz Introduction

https://takwabenaichagader.github.io/bigdata-session1-quiz/

Partie HDFS



Ouvrir le quiz HDFS

https://takwabenaichagader.github.io/bigdata-ch2-HDFS-quiz/

Synthèse : ce qu'il faut retenir



- **NameNode :** métadonnées et emplacements des blocs.
- **DataNodes :** stockage et transfert effectif des blocs.
- **Réplication :** plusieurs copies pour tolérer des défaillances.
- **Lire et écrire :** le client échange les données avec les DataNodes.